AgriLink

Input & Procurement Management

Complete Line-by-Line Technical Documentation

Spring Boot 4.0.6 • Java 21 • Spring Data JPA • MySQL

Module package: com.cts.agrilink.inputAndProcurementMangement

Generated: 2026-06-16

Author: Cognizant — AgriLink Team

Table of Contents

1. Project Overview & Architecture

AgriLink is a Spring Boot REST backend. The documented module — Input & Procurement Management — lets an agricultural platform maintain a catalog of farming inputs (seeds, fertilisers, pesticides, equipment) and manage the full lifecycle of farmer procurement requests against that catalog.

The two business domains

- AgriInput (the catalog): what items exist, their price, subsidised price, available stock, and availability status.
- InputRequest (the workflow): a farmer asks for a quantity of an input; the request then moves through Requested !’ Approved !’ Dispatched !’ Delivered, or is Cancelled.

Layered architecture

The code follows the classic Spring layered pattern. A request flows downward and the response flows back up:

- Controller — receives HTTP requests, validates the body, returns ResponseEntity. (AgriInputController, InputRequestController)
- Service — holds business rules: validation of categories/statuses, state-transition checks. (AgriInputService, InputRequestService)
- Repository — Spring Data JPA interfaces that talk to MySQL with zero hand-written SQL. (AgriInputRepository, InputRequestRepository)
- Model (Entity) — JPA-mapped classes that become database tables. (AgriInput, InputRequest, FarmerProfile)
- DTO — plain carriers for incoming/outgoing data, decoupling the API from the database schema.
- Exception — custom exceptions plus a single @RestControllerAdvice that turns them into clean JSON error responses.

Mental model of one call: HTTP !’ Controller (@Valid DTO) !’ Service (rules) !’ Repository (save/find) !’ MySQL, then back as JSON. Any error thrown is caught centrally by GlobalExceptionHandler and returned with the right HTTP status.

2. pom.xml — Maven Build Configuration

pom.xml is Maven's Project Object Model. It declares the project's identity, its parent, the libraries it depends on, and how it is built.

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0" ...>
3    <modelVersion>4.0.0</modelVersion>
4    <parent>
5      <groupId>org.springframework.boot</groupId>
6      <artifactId>spring-boot-starter-parent</artifactId>
7      <version>4.0.6</version>
8    </parent>
9    <groupId>com.cts</groupId>
10   <artifactId>agrilink</artifactId>
11   <version>0.0.1-SNAPSHOT</version>
12   <properties><java.version>21</java.version></properties>
```

Line 1: XML declaration — the file is XML, UTF-8 encoded.

Line 2–3: Root <project> element with the Maven 4.0.0 POM namespaces.

Line 3 (modelVersion): Always 4.0.0 for modern Maven; identifies the POM format version.

Line 5–8: Inherits from spring-boot-starter-parent 4.0.6. This parent fixes the versions of all Spring/Jackson/JUnit libraries so you don't specify them yourself, and sets sensible build defaults.

Line 11–13: This project's coordinates: group com.cts, artifact agrilink, version 0.0.1-SNAPSHOT (SNAPSHOT = under active development).

Line (java.version): Compiles and targets Java 21.

Dependencies

```
1  spring-boot-starter-data-jpa     // JPA + Hibernate ORM
2  spring-boot-starter-validation   // Bean Validation (@NotNull...)
3  spring-boot-starter-webmvc       // REST controllers, embedded Tomcat
4  spring-boot-devtools (runtime)   // hot reload during development
5  mysql-connector-j  (runtime)     // MySQL JDBC driver
6  lombok            (optional)     // generates getters/setters/builders
7  spring-boot-starter-test (test)  // JUnit 5, Mockito, AssertJ
8  h2                (test)         // in-memory DB for tests
```

data-jpa: Brings in Spring Data JPA and Hibernate so repositories work and entities map to tables.

validation: Enables @Valid and the jakarta.validation constraint annotations used in the DTOs.

webmvc: The web/REST stack with an embedded Tomcat server — lets the app serve HTTP without an external server.

devtools: Automatic restart and live reload while coding; runtime + optional so it is excluded from production jars.

mysql-connector-j: The JDBC driver that lets the app actually connect to MySQL; needed only at runtime.

lombok: Compile-time code generator. @Data, @Builder, @RequiredArgsConstructor etc. expand into real methods so the source stays short.

test deps + h2: Testing libraries plus an in-memory H2 database, so tests run fast without needing a real MySQL.

Build plugins

spring-boot-maven-plugin: Packages the app into a runnable 'fat' jar. It excludes Lombok from the final jar (Lombok is only needed at compile time).

maven-compiler-plugin: Registers Lombok as an annotation processor for both main and test compilation so the generated code appears during the build.

3. application.properties — Runtime Configuration

This file configures the running application: which database to use, how Hibernate behaves, and which port to listen on.

```
1  spring.application.name=agrilink
2  spring.datasource.url = jdbc:mysql://localhost:3306/agrilink
3  spring.datasource.username = root
4  spring.datasource.password =root
5
6  spring.jpa.show = true
7  spring.jpa.generate-ddl = true
8  spring.jpa.hibernate.ddl.auto = update
9  spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
10 spring.jpa.hibernate.naming.physical-strategy=...PhysicalNamingStrategyStandardImpl
11 server.port = 8082
```

Line 1: Logical name of the application (used in logs and, if added later, service discovery).

Line 2: JDBC URL — connect to a MySQL database called 'agrilink' on localhost port 3306.

Line 3–4: Database username and password (both 'root' here — fine for local dev, should be externalised for production).

Line 6: Show JPA/SQL activity (logging-related flag).

Line 7: Allow Hibernate to generate DDL (CREATE/ALTER table statements) from the entities.

Line 8 (ddl.auto = update): On startup Hibernate compares entities to the schema and ALTERs tables to match — it adds missing columns/tables but never drops data. Convenient for development.

Line 9: Tells Hibernate to speak MySQL's SQL dialect so generated SQL is MySQL-compatible.

Line 10: Physical naming strategy 'Standard' — keep the names exactly as written in @Table/@Column instead of converting camelCase to snake_case.

Line 11: The embedded server listens on port 8082 — so all URLs start with http://localhost:8082.

4. AgrilinkApplication.java — Bootstrap Class

The entry point. Running this class starts the whole Spring Boot application.

```
1  package com.cts.agrilink;
2
3  import org.springframework.boot.SpringApplication;
4  import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6  @SpringBootApplication
7  public class AgrilinkApplication {
8
9      public static void main(String[] args) {
10         SpringApplication.run(AgrilinkApplication.class, args);
11     }
12 }
```

Line 1: Package declaration — the root package. Spring scans this package and everything beneath it for components.

Line 3–4: Imports the two Spring Boot classes used below.

Line 6 (@SpringBootApplication): The master annotation. It combines three: @Configuration (this is a config class), @EnableAutoConfiguration (auto-wire beans based on the classpath — e.g. set up Tomcat, JPA), and @ComponentScan (find @Controller/@Service/@Repository in this package tree).

Line 7: The application class itself.

Line 9: Standard Java main method — the JVM starts here.

Line 10: Boots Spring: starts the embedded Tomcat, creates the application context, scans and wires all beans, and begins listening on port 8082.

5. Model Layer — JPA Entities

Entities are Java classes mapped to database tables. Each instance is one row; each field is one column. They use Lombok to avoid boilerplate.

Common Lombok / JPA annotations (used by all three entities)

@Entity: Marks the class as a JPA entity — Hibernate manages a table for it.

@Table(name=...): Names the database table explicitly.

@Data: Lombok — generates getters, setters, toString, equals and hashCode for every field.

@NoArgsConstructor: Generates an empty constructor (JPA requires one).

@AllArgsConstructor: Generates a constructor with every field as a parameter.

@Builder: Generates the fluent builder API, e.g. AgriInput.builder().name("X").build().

@Id: Marks the primary-key field.

@GeneratedValue(strategy = IDENTITY): The database auto-increments the key; you never set the id by hand.

@Column(name=...): Maps the field to a named column.

5.1 AgriInput.java — the catalog table

```
@Entity
@Table(name = "catalog")
@Data @NoArgsConstructor @AllArgsConstructor @Builder
public class AgriInput {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "inputId")  private Long inputId;
    @Column(name = "name")            private String name;
    @Column(name = "category")        private String category;
    @Column(name = "unit")            private String unit;
    @Column(name = "pricePerUnit")    private Double pricePerUnit;
    @Column(name = "subsidisedPrice") private Double subsidisedPrice;
    @Column(name = "availableStock")  private Integer availableStock;
    @Column(name = "status")          private String status;
}
```

Table 'catalog': Every catalog item is stored here.

inputId: Auto-generated primary key for the item.

name: Human-readable item name (e.g. 'Urea 50kg').

category: One of Seed / Fertiliser / Pesticide / Equipment (validated in the service).

unit: Unit of sale (kg, litre, packet, piece…).

pricePerUnit: Normal price per unit.

subsidisedPrice: Government/subsidised price (may be null when not subsidised).

availableStock: Current quantity in stock.

status: Available or OutOfStock (validated in the service).

5.2 InputRequest.java — the request/workflow table

```
@Entity
@Table(name = "request")
@Data @NoArgsConstructor @AllArgsConstructor @Builder
public class InputRequest {
    @Id @GeneratedValue(IDENTITY) Long requestId;
    Long    farmerId;          // who asked
    Long    inputId;           // which catalog item
    Integer quantityRequested; // how much
    LocalDate requestDate;     // when requested
    Long    assignedCentreId;  // fulfilling centre
```

```
    Double  actualPrice;       // agreed price
    String  status;            // lifecycle state
    String  approvedBy;
    String  dispatchedBy;
    LocalDate dispatchDate;
    LocalDate deliveredDate;
    String  receivedBy;
    String  cancellationReason;
}
```

Table 'request': Each procurement request is one row.

requestId: Auto-generated primary key.

farmerId / inputId: Foreign-key style references to the farmer and the catalog item (stored as plain Longs, not JPA relationships).

quantityRequested: Quantity the farmer wants.

requestDate: Date the request was created (LocalDate = date only, no time).

assignedCentreId: Distribution centre responsible for fulfilling it.

actualPrice: Final price decided at approval time.

status: The state-machine value: Requested / Approved / Dispatched / Delivered / Cancelled.

approvedBy / dispatchedBy / receivedBy: Audit fields — who performed each step.

dispatchDate / deliveredDate: When the item was dispatched and delivered.

cancellationReason: Why a request was cancelled (set only on cancel).

5.3 FarmerProfile.java — the farmer master table

```
@Entity
@Table(name = "farmerProfile")
@Data @NoArgsConstructor @AllArgsConstructor @Builder
public class FarmerProfile {
    @Id @GeneratedValue(IDENTITY) Long farmerId;
    Long      userId;          // link to login account
    String    name;
    LocalDate dateOfBirth;
    String    gender;
    String    nationalIdNumber;
    String    village; String district; String state;
    String    phone;
    String    bankAccountNumber;
    String    status;          // Active / Inactive / Verified
}
```

Purpose: Stores demographic and KYC details of each farmer.

farmerId: Primary key referenced by InputRequest.farmerId.

userId: Links the profile to a login/user account.

name, dateOfBirth, gender, nationalIdNumber: Identity fields.

village/district/state, phone: Address and contact details.

bankAccountNumber: For subsidy/payment disbursement.

status: Active / Inactive / Verified — the profile's standing.

Note: this entity is defined and creates its table, but the documented module does not yet expose controller/service endpoints for it — it supports the data model for future farmer-management features.

6. DTO Layer — Data Transfer Objects

DTOs carry data between the client and the API. They are separate from entities so the public API can validate input and stay decoupled from the database schema. All use @Data, @NoArgsConstructor, @AllArgsConstructor. Validation annotations come from jakarta.validation.

Validation annotations used

@NotBlank: String must be non-null and contain at least one non-whitespace character.

@NotNull: Value must not be null (used for numbers/dates).

@Positive: Number must be > 0.

@PositiveOrZero: Number must be >= 0.

Each message=... gives the exact error text returned to the client when the rule fails. These messages are collected by the GlobalExceptionHandler.

6.1 AgriInputRequestDTO — body for creating a catalog item

- name — @NotBlank 'Name is required'
- category — @NotBlank 'Category is required'
- unit — @NotBlank 'Unit is required'
- pricePerUnit — @NotNull + @Positive 'Price must be positive'
- subsidisedPrice — optional (no constraint, may be null)
- availableStock — @NotNull + @PositiveOrZero 'Stock cannot be negative'
- status — @NotBlank 'Status is required'

6.2 UpdateInputDTO — body for editing a catalog item

Same fields as above but without category and unit (those are not editable here): name, pricePerUnit (positive), subsidisedPrice (optional), availableStock (>= 0), status. All required fields validated identically.

6.3 UpdateStockDTO — body for adjusting stock only

- availableStock — @NotNull + @PositiveOrZero. The smallest DTO: it changes nothing but the stock count.

6.4 InputProcurementRequestDTO — body for submitting a request

- farmerId — @NotNull
- inputId — @NotNull
- quantityRequested — @NotNull + @Positive
- requestDate — @NotNull (LocalDate)
- assignedCentreId — @NotNull
- actualPrice — optional (set later at approval)

6.5 UpdateInputRequestDTO — body for editing a pending request

All fields optional (used for partial updates): quantityRequested (@Positive when present), assignedCentreId, actualPrice. The service only updates fields that are non-null.

6.6 ApproveRequestDTO — body for the approve step

- assignedCentreId — @NotNull
- actualPrice — @NotNull
- approvedBy — @NotBlank

6.7 DispatchRequestDTO — body for the dispatch step

- dispatchedBy — @NotBlank
- dispatchDate — @NotNull

6.8 DeliverRequestDTO — body for the deliver step

- deliveredDate — @NotNull
- receivedBy — @NotBlank

6.9 CancelRequestDTO — body for cancelling

- reason — @NotBlank 'Reason is required'

6.10 MessageResponseDTO — the universal response

```
@Data @NoArgsConstructor @AllArgsConstructor
public class MessageResponseDTO {
    private String message;
}
```

Purpose: A one-field wrapper { "message": "..." }. Every write operation and every error returns one of these, so clients always get a consistent JSON shape.

7. Repository Layer — Spring Data JPA

Repositories are interfaces. Spring Data JPA generates the implementation at runtime — no SQL or boilerplate to write. By extending JpaRepository<Entity, IdType> they instantly get findAll, findById, save, deleteById, existsById, and more.

7.1 AgriInputRepository

```
@Repository
public interface AgriInputRepository
        extends JpaRepository<AgriInput, Long> {
    List<AgriInput> findByCategoryIgnoreCase(String category);
    List<AgriInput> findByStatusIgnoreCase(String status);
}
```

@Repository: Marks this as a data-access bean and enables JPA exception translation.

extends JpaRepository<AgriInput, Long>: AgriInput is the entity, Long is its primary-key type. Inherits all standard CRUD methods.

findByCategoryIgnoreCase: A derived query. Spring reads the method name and builds 'WHERE category = ? (case-insensitive)' automatically.

findByStatusIgnoreCase: Same idea — finds all items with a given status, ignoring case.

7.2 InputRequestRepository

```
@Repository
public interface InputRequestRepository
        extends JpaRepository<InputRequest, Long> {
    List<InputRequest> findByFarmerId(Long farmerId);
    List<InputRequest> findByStatusIgnoreCase(String status);
    List<InputRequest> findByAssignedCentreId(Long centreId);
}
```

findByFarmerId: All requests submitted by one farmer.

findByStatusIgnoreCase: All requests in a given lifecycle state, case-insensitive.

findByAssignedCentreId: All requests assigned to a particular distribution centre.

8. Exception Layer

Two custom exceptions express specific failure types, and one global handler converts every exception into a clean JSON response with the correct HTTP status code.

8.1 ResourceNotFoundException

```
public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String message) {
        super(message);
    }
}
```

extends RuntimeException: Unchecked — can be thrown without declaring it. Used when an id does not exist in the database. Maps to HTTP 404.

8.2 StateConflictException

```
public class StateConflictException extends RuntimeException {
    public StateConflictException(String message) {
        super(message);
    }
}
```

Purpose: Thrown when an operation is illegal for the current lifecycle state (e.g. dispatching a request that was never approved). Maps to HTTP 409 Conflict.

8.3 GlobalExceptionHandler

```
@RestControllerAdvice
public class GlobalExceptionHandler {
  @ExceptionHandler(ResourceNotFoundException.class)
  ... -> 404 NOT_FOUND, body = new MessageResponseDTO(ex.getMessage())

  @ExceptionHandler(MethodArgumentNotValidException.class)
  ... collect field errors -> 400 BAD_REQUEST

  @ExceptionHandler(StateConflictException.class)
  ... -> 409 CONFLICT

  @ExceptionHandler(IllegalArgumentException.class)
  ... -> 400 BAD_REQUEST
}
```

@RestControllerAdvice: A global interceptor. Any exception thrown by any controller is routed here instead of leaking a raw stack trace to the client.

handleNotFound: Catches ResourceNotFoundException and returns HTTP 404 with the message in a MessageResponseDTO.

handleValidation: Catches @Valid failures (MethodArgumentNotValidException). It streams over every failed field, formats each as 'field: message', joins them with commas, and returns HTTP 400.

handleConflict: Catches StateConflictException and returns HTTP 409 — the correct code for an illegal state transition.

handleBadRequest: Catches IllegalArgumentException (e.g. invalid category/status from the service) and returns HTTP 400.

Net effect: clients always receive { "message": "..." } with a meaningful HTTP status, never an internal error page.

9. Service Layer — AgriInputService

Holds the business logic for the catalog. Annotated @Service (a Spring bean) and @RequiredArgsConstructor (Lombok generates a constructor for the final fields, so the repository is injected automatically — constructor injection).

```
@Service @RequiredArgsConstructor
public class AgriInputService {
    private final AgriInputRepository agriInputRepository;
```

private final repository: The single dependency. Because it is final and the class is @RequiredArgsConstructor, Spring injects it through the generated constructor.

getAllInputs()

Body: return agriInputRepository.findAll(); — returns every catalog item.

getInputById(Long inputId)

```
return agriInputRepository.findById(inputId)
    .orElseThrow(() -> new ResourceNotFoundException(
        "Input not found with ID: " + inputId));
```

findById: Returns an Optional<AgriInput>.

.orElseThrow: If the Optional is empty (no such id), throw ResourceNotFoundException !' becomes a 404. Otherwise return the found item.

getInputsByCategory(String category)

```
List<String> valid = List.of("Seed","Fertiliser","Pesticide","Equipment");
if (valid.stream().noneMatch(c -> c.equalsIgnoreCase(category)))
    throw new IllegalArgumentException("Invalid category: " + category);
return agriInputRepository.findByCategoryIgnoreCase(category);
```

valid list: The whitelist of acceptable categories.

noneMatch(...equalsIgnoreCase): If the supplied category matches none of the four (ignoring case), it is invalid !' throw IllegalArgumentException !' 400.

return: Otherwise delegate to the repository's case-insensitive finder.

getInputsByStatus(String status)

Logic: Identical pattern but the whitelist is { Available, OutOfStock }. Invalid status !' 400; valid !' findByStatusIgnoreCase.

addInput(AgriInputRequestDTO dto)

```
AgriInput input = AgriInput.builder()
    .name(dto.getName()).category(dto.getCategory())
    .unit(dto.getUnit()).pricePerUnit(dto.getPricePerUnit())
    .subsidisedPrice(dto.getSubsidisedPrice())
    .availableStock(dto.getAvailableStock())
    .status(dto.getStatus()).build();
agriInputRepository.save(input);
return new MessageResponseDTO("Input catalog item created successfully");
```

builder(): Maps each DTO field onto a new AgriInput entity using the Lombok builder.

save(input): Inserts the new row (id auto-generated).

return: A success message. (The controller wraps this in HTTP 201 Created.)

updateInput(Long inputId, UpdateInputDTO dto)

```
AgriInput input = findById(inputId).orElseThrow(... 404 ...);
input.setName(dto.getName());
input.setPricePerUnit(dto.getPricePerUnit());
input.setSubsidisedPrice(dto.getSubsidisedPrice());
input.setAvailableStock(dto.getAvailableStock());
input.setStatus(dto.getStatus());
```

```
agriInputRepository.save(input);
return new MessageResponseDTO("Input catalog item updated successfully");
```

Load-or-404: First fetch the existing item; throw 404 if missing.

setters: Overwrite the editable fields from the DTO (note: category and unit are not changed).

save: Because the entity already has an id, JPA issues an UPDATE, not an INSERT.

updateStock(Long inputId, UpdateStockDTO dto)

Logic: Load-or-404, then set only availableStock and save. Returns 'Stock updated successfully'. A focused operation for inventory adjustments.

deleteInput(Long inputId)

```
if (!agriInputRepository.existsById(inputId))
    throw new ResourceNotFoundException(... 404 ...);
agriInputRepository.deleteById(inputId);
return new MessageResponseDTO("Input catalog item deleted successfully");
```

existsById guard: Confirms the row exists first, so deleting a missing id returns a clean 404 rather than failing silently.

deleteById + return: Removes the row and returns a success message.

10. Service Layer — InputRequestService

The heart of the procurement workflow. It has two dependencies — the request repository and the catalog repository — both injected via the Lombok-generated constructor.

```
@Service @RequiredArgsConstructor
public class InputRequestService {
    private final InputRequestRepository inputRequestRepository;
    private final AgriInputRepository  agriInputRepository;
```

Read methods

getAllRequests(): findAll() — every request.

getRequestById(id): findById + orElseThrow !’ 404 if missing.

getRequestsByFarmer(farmerId): All requests of one farmer.

getRequestsByCentre(centreId): All requests for one centre.

getRequestsByStatus(status): Validates against { Requested, Approved, Dispatched, Delivered, Cancelled }; invalid !’ 400; valid !’ findByStatusIgnoreCase.

submitRequest(InputProcurementRequestDTO dto)

```
AgriInput input = agriInputRepository.findById(dto.getInputId())
    .orElseThrow(... 404 'Input not found' ...);
InputRequest request = InputRequest.builder()
    .farmerId(dto.getFarmerId()).inputId(dto.getInputId())
    .quantityRequested(dto.getQuantityRequested())
    .requestDate(dto.getRequestDate())
    .assignedCentreId(dto.getAssignedCentreId())
    .actualPrice(dto.getActualPrice())
    .status("Requested").build();
inputRequestRepository.save(request);
return new MessageResponseDTO("Input request submitted successfully");
```

Validate the input exists: Before creating a request the referenced catalog item must exist, otherwise 404.

builder + status 'Requested': Builds the new request and forces its initial state to 'Requested' — the workflow always starts here regardless of input.

save + return: Persists the new request and returns success (controller !’ 201 Created).

updateRequest(Long requestId, UpdateInputRequestDTO dto) — partial update

```
InputRequest request = findById(requestId).orElseThrow(... 404 ...);
if (dto.getQuantityRequested() != null) request.setQuantityRequested(...);
if (dto.getAssignedCentreId()  != null) request.setAssignedCentreId(...);
if (dto.getActualPrice()       != null) request.setActualPrice(...);
inputRequestRepository.save(request);
```

Null checks: Each field is only overwritten if the client actually sent it. This is a PATCH-style partial update — omitted fields keep their current value.

approveRequest(Long requestId, ApproveRequestDTO dto)

```
InputRequest request = findById(requestId).orElseThrow(... 404 ...);
if (!"Requested".equals(request.getStatus()))
    throw new StateConflictException(
        "Request cannot be approved. Current status: " + status);
request.setStatus("Approved");
request.setAssignedCentreId(dto.getAssignedCentreId());
request.setActualPrice(dto.getActualPrice());
request.setApprovedBy(dto.getApprovedBy());
inputRequestRepository.save(request);
```

State guard: Only a request currently in 'Requested' may be approved. Anything else throws StateConflictException !’ 409.

Transition: Sets status to 'Approved' and records the centre, agreed price and approver.

dispatchRequest(Long requestId, DispatchRequestDTO dto)

Guard: Status must be 'Approved', else 409.

Transition: Sets status 'Dispatched', records dispatchedBy and dispatchDate, then saves.

deliverRequest(Long requestId, DeliverRequestDTO dto)

Guard: Status must be 'Dispatched', else 409.

Transition: Sets status 'Delivered', records deliveredDate and receivedBy, then saves.

cancelRequest(Long requestId, CancelRequestDTO dto)

```
if ("Dispatched".equals(status) || "Delivered".equals(status))
    throw new StateConflictException(
        "Cannot cancel request after dispatch...");
request.setStatus("Cancelled");
request.setCancellationReason(dto.getReason());
```

Guard: A request can be cancelled while Requested or Approved, but NOT once it is Dispatched or Delivered (goods already in motion) !’ 409.

Transition: Sets status 'Cancelled' and stores the reason.

deleteRequest(Long requestId)

Logic: existsById guard !’ 404 if missing, then deleteById and return success. Removes the request record entirely.

11. Controller Layer — AgriInputController

Exposes the catalog as REST endpoints. @RestController means every method returns data serialised to JSON. @RequestMapping sets the common URL prefix. The service is injected via the Lombok constructor.

```
@RestController
@RequestMapping("/agriLink/procurementManagement/inputs")
@RequiredArgsConstructor
public class AgriInputController {
    private final AgriInputService agriInputService;
```

@RestController: Combines @Controller + @ResponseBody — return values become the HTTP response body (JSON).

@RequestMapping(prefix): All endpoints below start with /agriLink/procurementManagement/inputs.

The eight endpoints

GET / (getAllInputs): Returns 200 with the full catalog list.

GET /{inputId}: @PathVariable binds the URL id; returns the item or 404.

GET /category/{category}: Returns items in that category (400 if category invalid).

GET /status/{status}: Returns items with that status (400 if status invalid).

POST / (addInput): @Valid @RequestBody validates the DTO; returns 201 Created + message.

PUT /{inputId} (updateInput): @Valid body updates the item; 200 + message, or 404.

PUT /{inputId}/stock (updateStock): Adjusts only the stock; 200 + message.

DELETE /{inputId}: Deletes the item; 200 + message, or 404.

ResponseEntity.ok(...) / .status(CREATED): Wrap the body with the explicit HTTP status code.

12. Controller Layer — InputRequestController

Exposes the request workflow under the prefix /agriLink/procurementManagement/input-requests. Same patterns: @RestController, constructor-injected service, @Valid bodies, @PathVariable ids.

Read endpoints

GET /: All requests.

GET /{requestId}: One request or 404.

GET /farmer/{farmerId}: Requests of a farmer.

GET /status/{status}: Requests by lifecycle state (400 if invalid).

GET /centre/{centreId}: Requests for a centre.

Write / workflow endpoints

POST /: submitRequest — 201 Created.

PUT /{requestId}: updateRequest — partial edit of a pending request.

PUT /{requestId}/approve: approveRequest — Requested !’ Approved.

PUT /{requestId}/dispatch: dispatchRequest — Approved !’ Dispatched.

PUT /{requestId}/deliver: deliverRequest — Dispatched !’ Delivered.

PUT /{requestId}/cancel: cancelRequest — cancel while not yet dispatched.

DELETE /{requestId}: deleteRequest — remove the record.

Each endpoint simply delegates to the matching service method and wraps the result in ResponseEntity. All validation and state rules live in the service; all error formatting lives in the GlobalExceptionHandler.

13. Request Lifecycle (State Machine)

InputRequest.status moves through a strict set of transitions enforced by the service guards:

```
                approve            dispatch           deliver
Requested  ----------->  Approved  --------->  Dispatched  -------->  Delivered
    |                       |
    | cancel                | cancel
    v                       v
Cancelled               Cancelled

Rules enforced in InputRequestService:
 - approve  : only from Requested      else 409 StateConflict
 - dispatch : only from Approved       else 409
 - deliver  : only from Dispatched     else 409
 - cancel   : NOT allowed once Dispatched or Delivered  else 409
```

- Forward-only: you cannot skip a step (e.g. deliver before dispatch).
- Cancellation is allowed early (Requested/Approved) but blocked once goods are dispatched.
- Every illegal transition produces HTTP 409 Conflict with an explanatory message.

14. End-to-End Request Flow (Worked Example)

Following a single farmer procurement from start to finish:

- 1) Admin creates a catalog item: POST /inputs with AgriInputRequestDTO !’ 201, row in 'catalog'.
- 2) Farmer submits a request: POST /input-requests with InputProcurementRequestDTO !’ service checks the inputId exists, saves a row in 'request' with status='Requested' !’ 201.
- 3) Officer approves: PUT /input-requests/{id}/approve !’ guard checks status=='Requested', sets Approved + approvedBy + actualPrice.
- 4) Warehouse dispatches: PUT /{id}/dispatch !’ guard checks Approved, sets Dispatched + dispatchedBy + dispatchDate.
- 5) Farmer receives: PUT /{id}/deliver !’ guard checks Dispatched, sets Delivered + receivedBy + deliveredDate.
- Alternative: at steps 2–3 the request may be cancelled via PUT /{id}/cancel; after dispatch cancellation is rejected with 409.

Throughout, validation errors return 400, missing ids return 404, and illegal state changes return 409 — all as { "message": "..." } thanks to the GlobalExceptionHandler. Successful writes return { "message": "...successfully" }.